

答案解析与参考标准

一、选择题解析

1. **C** (测试的目的是发现错误, 而非证明无错,)
2. **A** (总体设计/概要设计关注宏观结构)
3. **D** (UML中标准可见性为public, private, protected, package/friendly, 没有 global)
4. **C** (基线变更需CCB授权)
5. **D** (DFD包含实体、过程、数据流、存储, 联系是ER图的概念)
6. **D** (封装重用通常指黑盒复用, "类构件的重用"通常指继承、聚合等机制。直接修改内部代码违反了"对修改关闭"原则, 不属于规范的构件重用)
7. **B** (判定表适合处理多条件的复杂逻辑组合)
8. **B** (软件成本是不断增加的, 而非降低)
9. **B** (螺旋模型的核心特点是风险分析)

二、填空题解析

1. **等价类划分** (Equivalence Partitioning)
2. **回归** (Regression)
3. **系统化** (systematic)、**可量化** (quantifiable)
4. **结构化** (Structured)
5. **评审** (Review)、**软件测试** (Testing)
6. **变更** (Change) 或 **配置项** (Configuration Items)

三、简答题解析

1. 体系结构风格-

- **定义:** 描述特定应用领域中系统组织方式的惯用模式, 提供预定义的子系统、职责及交互规则。
- **举例:**
 1. **分层风格 (Layered):** 系统被组织成层次结构, 每一层为上层提供服务, 并使用下层服务 (如 OSI 模型)。
 2. **客户/服务器风格 (Client/Server):** 系统分为请求服务的客户端和提供服务的服务器, 分布在不同节点。
 3. **MVC 风格:** 将系统分为模型 (数据)、视图 (显示) 和控制器 (交互) 三部分, 实现关注点分离。

2. 违反职业规范的行为

- **答案要点 (任选4点):**
 1. 为个人利益窃取公司或客户的数据。
 2. 恶意破坏或修改他人的程序代码、文件。
 3. 制作、传播计算机病毒或恶意软件。
 4. 在未授权的情况下使用客户的资源。
 5. 交付明知有严重缺陷的软件而不告知客户。

3. XP 重构,

- **定义：** 在不改变代码外在行为的前提下，对代码内部结构进行修改，以提高其可理解性和可维护性。
- **方法与解决问题：**
 1. **提炼函数 (Extract Method)：** 解决“过长函数” (Long Method) 问题，将大函数拆分为小函数。
 2. **函数改名 (Rename Method)：** 解决“函数名含义不清”的问题，使其具有自解释性。
 3. **内联函数 (Inline Method)：** 解决“过度委托”或函数体过于简单的问题，减少不必要的调用开销。

4. 软件配置与 SCM 任务

- **软件配置：** 指软件生命周期中产生的所有信息项的集合，包括程序（源码/可执行）、文档和数据。
- **五项任务：**
 1. **配置标识 (Identification)：** 识别和命名配置项。
 2. **版本控制 (Version Control)：** 管理配置项的不同版本。
 3. **变更控制 (Change Control)：** 审批和管理对基线的变更 (CCB) 。
 4. **配置审计 (Configuration Auditing)：** 验证配置项是否符合要求。
 5. **状态报告 (Status Reporting)：** 记录和报告配置项的状态及变更历史。

5. 软件可靠性 vs 可用性

- **可靠性 (Reliability)：** 在规定条件下、规定时间内，软件系统无故障运行的概率。侧重于“不坏”。
- **可用性 (Usability/Availability)：** *注：课件中将 Usability 译为可用性/易用性，Availability 译为可用性。根据题意“成功运行的概率”，这里指 Availability。*
 - **Availability (可用性)：** 系统在任意给定时刻可以正常工作的概率 (Point in time) 。
- **区别：** 可靠性强调一段时间内的持续无故障；可用性强调在某个时刻系统是“在线”的。一个系统经常崩溃但重启极快，可能可靠性低但可用性高。

6. 集成策略优缺点

- **非渐增式 (Big Bang)：**
 - 优点：测试次数少，看似节省时间。
 - 缺点：错误定位极其困难，容易遗漏接口错误，不仅不节约反而可能延误工期。
- **渐增式 (Incremental)：**
 - 优点：容易定位错误（因为每次只增加一个模块），测试更彻底，允许并行开发。
 - 缺点：需要编写驱动程序 (Driver) 或桩程序 (Stub)，测试次数较多，机器机时消耗大。

四、设计题解析与 Mermaid 代码

1. 设计原则 (Law of Demeter)

- **(1) 违反原则：迪米特法则 (Law of Demeter, LoD) 或 最少知识原则。**
 - **内容：** 一个对象应该对其他对象有最少的了解。不要和“陌生人”说话，只与直接的朋友（成员变量、参数、创建的对象）交互。题目中 **Search** 直接操作遍历结果，或者调用链过长，违反了松耦合。
- **(2) 改进后的类图：** 应通过封装，由中间类委托调用。

```
classDiagram
    class Client {
        +search()
    }
```

```
class SearchEngine {
    +search(query) Results
}
class Indexer {
    -indexData
    +lookup()
}
class ResultPrinter {
    +print(Results)
}

Client --> SearchEngine : uses
SearchEngine --> Indexer : uses
SearchEngine --> ResultPrinter : uses
%% 改进点: Client 不直接依赖 Indexer 或 ResultPrinter, 由 SearchEngine 统一调度
```

2. 基本路径测试

- (1) 环形复杂度 $V(G)$:
 - 判定节点数 $P = 2$ (if age<18, if isMember)。
 - $V(G) = P + 1 = 3$ 。
- (2) 独立路径:
 - Path 1: 1 -> 3 -> 5 (age >= 18, !isMember)
 - Path 2: 1 -> 2 -> 3 -> 5 (age < 18, !isMember)
 - Path 3: 1 -> 3 -> 4 -> 5 (age >= 18, isMember)
 - (注: Path 4: 1 -> 2 -> 3 -> 4 -> 5 也是一条路径, 但基本路径集只需覆盖所有边即可, $V(G)=3$ 意味着3条线性无关路径即可构成基底, 通常取前3条)
- (3) 测试用例:
 1. Path 1: age = 20, isMember = false -> Expected: 100
 2. Path 2: age = 10, isMember = false -> Expected: 50
 3. Path 3: age = 20, isMember = true -> Expected: 80

3. 电梯系统 (Use Case & Class)

用例图:

类图:

```
classDiagram
    class Elevator {
        -currentFloor: int
        -state: Enum
        +moveUp()
        +moveDown()
        +openDoor()
        +closeDoor()
    }
    class Button {
        -isPressed: bool
    }
```

```

        +press()
    }
    class Scheduler {
        +dispatch()
        +receiveRequest()
    }
    class Floor {
        -floorNum: int
    }

    Scheduler "1" --> "1..*" Elevator : controls
    Button "1..*" --> "1" Scheduler : signals
    Elevator "1" --o "1..*" Floor : serves

```

4. ATM 取款 (Sequence & State)

顺序图:

```

sequenceDiagram
    participant U as User
    participant ATM as ATM_GUI
    participant S as BankServer
    participant A as Account

    U->>ATM: Insert Card & PIN
    ATM->>S: Verify(Card, PIN)
    S-->>ATM: Valid
    U->>ATM: Input Amount
    ATM->>S: RequestWithdraw(Amount)
    S->>A: CheckBalance()
    alt Balance OK
        A-->>S: Sufficient
        S->>A: Deduct(Amount)
        S-->>ATM: Approve
        ATM-->>U: Dispense Cash & Eject Card
    else Insufficient
        A-->>S: Insufficient
        S-->>ATM: Deny
        ATM-->>U: Show Error
    end
end

```

状态图 (Account):

```

stateDiagram-v2
    [*] --> Idle: 插入卡片
    Idle --> Verifying: 输入密码
    Verifying --> Idle: 密码错误
    Verifying --> LoggedIn: 密码正确

```

```
state LoggedIn {  
  [*] --> Selecting  
  Selecting --> Checking: 输入金额  
  Checking --> Selecting: 余额不足  
  Checking --> Updating: 余额充足  
  Updating --> Selecting: 扣款完成  
}
```

LoggedIn --> Idle: 退卡/完成

Idle --> Locked: 密码错误3次